1.人工智能概述
概念
产生和发展
不同学派
例题
2.知识表示与谓词
确定推理
不确定性推理
演绎推理与归纳推理的区别
一阶谓词逻辑
3.知识表示与推理
产生式表示法
语义网络表示法
框架表示法
4.谓词与自然演绎推理
常用等价式：
永真蕴含式：
自然演绎推理
5.归结演绎推理
子句
化简子句集(9个步骤)
鲁滨逊归结原理
置换
合一
定理证明
问题求解
归结演绎推理的方法
命题逻辑的归结演绎推理
谓词逻辑的归结演绎推理
主观贝叶斯?
9.状态空间
搜索概述
盲目搜索
智能搜索
状态空间法
10.问题规约
分解
等价变换
有解节点&不可解节点
解树
11.启发式搜索
A算法
A*算法
12.与或树的搜索
一般搜索思想
一般搜索过程
与或树的广搜
与或树的深搜
解树的代价
希望树
13.博弈树
概念
博弈树
博弈树的特点
极大极小分析法
α-β剪枝技术

15.机器学习朴素贝叶斯与决策树
机器学习
回归
分类
有监督学习
半监督学习
无监督学习(UnsupervisedLearning)
朴素贝叶斯
贝叶斯定理
朴素贝叶斯分类过程
决策树

1.人工智能概述

概念

人工智能：Artificial intelligence (ai)

- 能力角度：用人工方法在机器上实现的智能（也称机器智能）
- 学科角度：研究如何构造智能机器或智能系统，以模拟、延申和扩展人类智能

强人工智能

> 有可能制造出真正能推理和解决问题的智能机器，并且它将被认为是有知觉的，有自我意识的，分为类人的人工智能、非类人的人工智能

弱人工智能

> 不可能制造出真正能推理和解决问题的智能机器，这些机器只不过看起来像是智能的，但是并不真正拥有智能，也不会有自主意识

人工智能的最终目标

> 探讨智能形成的基本机理，研究利用自动机模拟人的思维过程

近期目标

> 研究如何才能使计算机去做那些靠人的智力才能去做的工作

图灵测试

> 如果机器的回答超过30%能让测试主持人误认为自己是人，则认为机器拥有了智能

中文屋（反驳图灵测试）

> 只要计算机拥有了适当的程序，理论上就可以说计算机拥有它的认知状态以及可以像人一样地进行理解活动。
>
> “中文房间”最早由美国哲学家John Searle于20世纪80年代初提出。这个实验要求你想象一位只说英语的人身处一个房间之中，这间房间除了门上有一个小窗口以外，全部都是封闭的。他随身带着一本写有中文翻译程序的书。房间里还有足够的稿纸、铅笔和橱柜。写着中文的纸片通过小窗口被送入房间中。根据Searle，房间中的人可以使用他的书来翻译这些文字并用中文回复。虽然他完全不会中文，Searle认为通过这个过程，房间里的人可以让任何房间外的人以为他会说流利的中文。

产生和发展

产生

> 1956年，达特茅斯会议上正式提出人工智能
>
> **麦卡锡**召集，包括**明斯基**、**罗切斯特**、**香农**、莫尔、塞缪尔、塞尔弗里奇、索罗蒙夫、纽厄尔、西蒙

不同学派

- 符号主义
  - 功能模拟，构造能模拟大脑功能的智能系统。“鸟飞”
- 联结主义
  - 结构模拟，构造模拟大脑结构的神经网络系统。“飞鸟”
- 行为主义
  - 行为模拟，构造具有进化能力的智能系统。“由猿到人”

例题

自然语言处理属于?

1.
- A 符号智能
- B 计算智能
- C 机器学习
- D 机器感知

2.知识表示与谓词

确定推理

推理所用的知识和推出的结论都是可以精确表示的，其真值要么为真，要么为假，不会有第三种情况出现。

产生式推理、自然演绎推理、归结演绎推理

不确定性推理

推理时所用的知识不都是确定的，推出的结论也不完全是确定的，其值会介于真与假之间。

可信度推理、主观Bayes推理、证据理论、模糊推理、概率推理

演绎推理与归纳推理的区别

> 演绎推理（一般到个别）
> 是在已知领域内的一般性知识的前提下，通过演绎求解一个具体问题或者证明一个结论的正确性。它所得出的结论实际上早已蕴含在一般性知识的前提中，演绎推理只不过是将已有事实揭露出来，因此它 不能增殖新知识 。
>
> 归纳推理（个别到一般）
> 所推出的结论是没有包含在前提内容中的。这种由个别事物或现象推出一般性知识的过程， 是 增殖新知识 的过程。

一阶谓词逻辑

- 命题：一个陈述句称为断言。凡有真假意义的断言称为命题。
- 论域(个体域)：是由所讨论对象之全体构成的非空集合。即 **个体变元的取值范围**，可以是无限集。
- 谓词：在谓词逻辑中，命题是用谓词来表示的。由谓词名和个体(可以是常量、变元、函数、谓词)组成。

> 一般形式为$P(x_1,x_2,...,x_n)$
>
> - **老张是教师 Teacher(Zhang)**
> - **5<3 Greater(5,3)**
>
> 若x_i都是个体常量、变元或函数，则称P为一阶谓词，若xi又是一阶谓词，则称P为二　阶谓词，以此类推。

谓词与函数的区别：

谓词是Dn到{T，F}的映射，函数是Dn到D的映射。谓词的真值是T和F，函数的值（无真值）是D中的元素。谓词可独立存在，函数只能作为谓词的个体

- 连接词：

  ¬：　“非”或者“否定”。表示对其后面的命题的否定
  ∨：“析取”。表示所连结的两个命题之间具有“或”的关系
  ∧：“合取”。 表示所连结的两个命题之间具有“与”的关系。
  →：　“条件”或“蕴含”。表示“若...则...”的语义。读作“如果P，则Q”，其中，P称为条件的前件，Q称为条件的后件。
  ↔：称为“双条件”。它表示“当且仅当”的语义。即读作“P当且仅当Q”。例如，对命题P和Q，P↔Q表示“P当且仅当Q”

- 量词

  ∀：全称量词。意思是“所有的”、“任一个”

  ∃：存在量词，意思是“至少有一个”、“存在有”

> 优先级：¬，∧，∨，→，↔

- 辖域：指位于量词后面的单个谓词或者用括弧括起来的合式公式
- 约束变元：辖域内与量词中同名的变元称为约束变元
- 自由变元：不受约束的变元称为自由变元

> (∀x)(P(x，y)→Q(x，y))∨R(x，y)
> 其中，(P(x，y)→Q(x，y))是(∀x)的辖域
> 辖域内的变元x是受(∀x)约束的变元
> R(x，y)中的x和所有的y都是自由变元

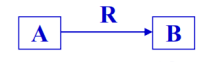
3.知识表示与推理

产生式表示法

IF P THEN Q

语义网络表示法

> 结点：实体
>
> 弧：语义联系
>
> 语意基元：语义网络中最基本的语义单元，（结点1，弧，结点2）
>
> 基本网元：一个语义基元对应的有向图，时语义网络中最基本的结构单元



图：基本网元结构

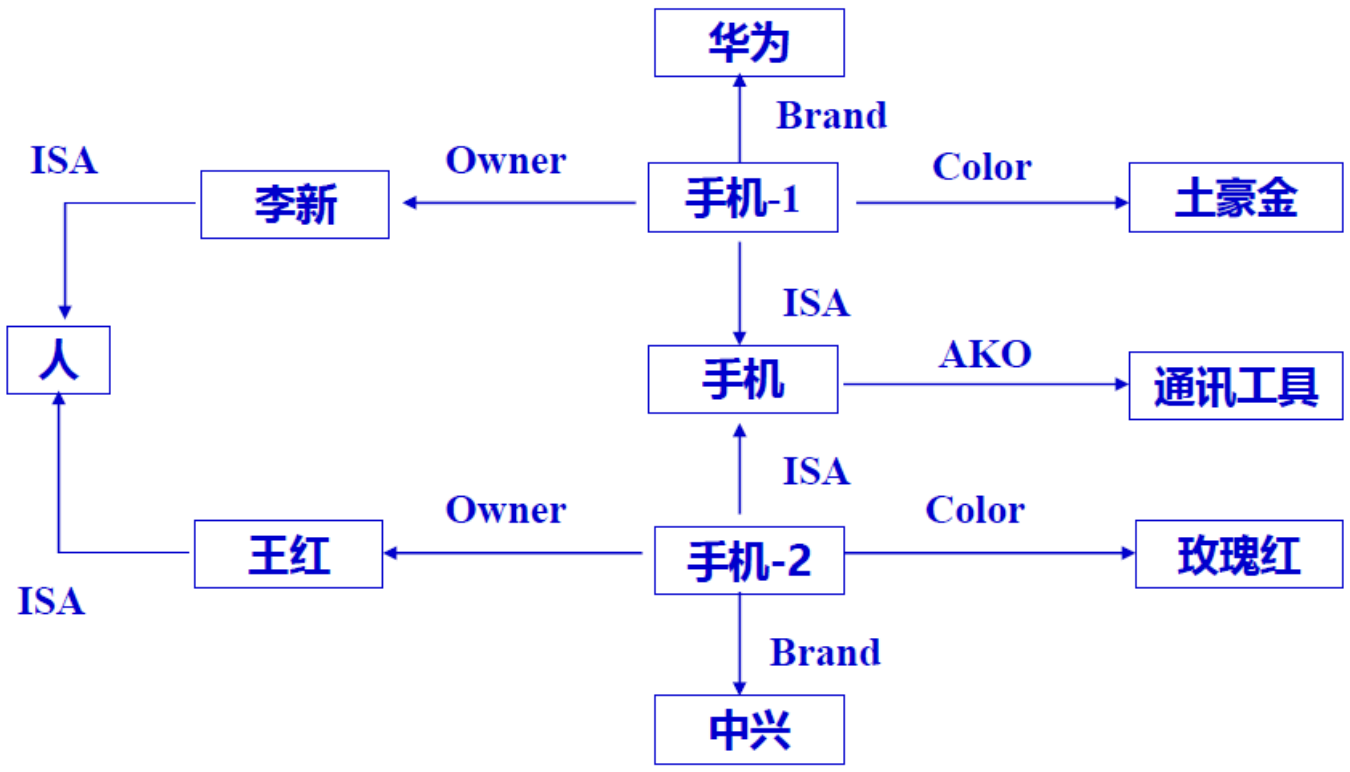


图：鸵鸟的基本网元

基本的语义关系

实例关系ISA，分类关系AKO，成员关系A-Menber_of，属性关系(Have/Can/Age)，包含关系Part-of，时间关系(Before/After)，位置关系Located-~，相近关系(Similar-to/Near-to)

例2.10：李新的手机牌子是“华为”、土豪金色。
王红的手机牌子是“中兴”、玫瑰红色。
解：李新和王红的手机均属于具体概念,可增加“手机” 这个抽象概念。

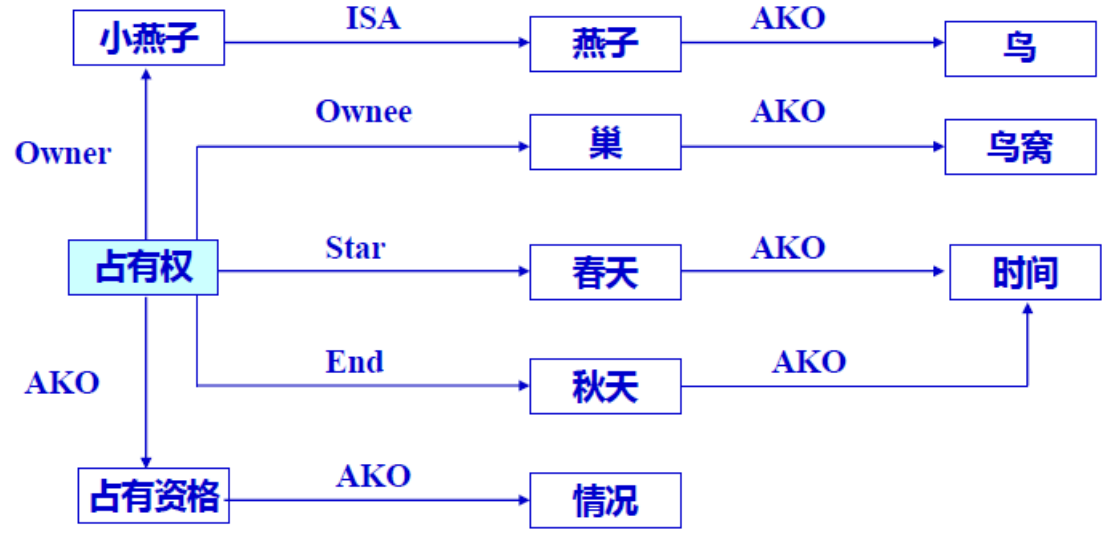


26

增加情况和动作节点

表示方法：为了描述复杂的情况和动作，西蒙提出了增加情况和动作结点的描述方法。

例2.11：用语义网络表示：
“小燕子这只燕子从春天到秋天一直占有一个巢”
解：需要设立一个占有权结点，表示占有物和占有时间等。



28

框架表示法

基本结构：

<框架名>

槽名1：侧面名1_1 值1_{11}，值1_{12}，…

侧面名1_2 值1_{21}，值1_{22}，…

：

槽名2：侧面名2_1 值2_{11}，值2_{12}，…

侧面名2_2 值2_{21}，值2_{22}，…

：

：

：

槽名n：侧面名n_1 值n_{11}，值n_{12}，…

侧面名n_2 值n_{21}，值n_{22}，…

：

侧面名n_m 值n_{m1}，值n_{m2}，…

框架的槽值和侧面值即可以是数字、字符串、布尔值，也可以是一个给定的操作，另一个框架的名字。

操作：在推理过程中，调用操作，实现对侧面值的动态计算或修改。

框架：在推理过程在，调用该框架，实现框架之间的联系。

实例：

例2.14 一个直接描述硕士生有关情况的框架

```
Frame <MASTER>                                          \\框架名
  Name：Unit（Last-name，First-name）                    \\姓名
  Sex：Area（male，female）                              \\性别
       Default： male                                    \\缺省值
Age：Unit（Years）                                      \\年龄
Major：Unit（Major）                                    \\专业
Field：Unit（Field）                                    \\方向
Advisor：Unit（Last-name，First-name）                  \\导师
Project：Area（National，Provincial，Other）            \\科研项目
         Default：National
Paper：Area（SCI，EI，Core，General）                   \\论文
       Default：Core
Address：<S-Address>                                    \\住址
Telephone：Home Unit（Number）                          \\电话
           Mobile Unit（Number）
```

Unit(单位)：填写槽值或侧面值的格式

Area(范围)：所填槽值的范围

Default(缺省值)

<框架名>

如，地址的框架名<S-Address>

4.谓词与自然演绎推理

永真性&永假性

常用等价式：

(1) **双重否定律** $\neg\neg P \Leftrightarrow P$

(2) **交换律** $(P\vee Q)\Leftrightarrow(Q\vee P)$，$(P\wedge Q)\Leftrightarrow(Q\wedge P)$

(3) **结合律** $(P\vee Q)\vee R\Leftrightarrow P\vee(Q\vee R)$

$(P\wedge Q)\wedge R\Leftrightarrow P\wedge(Q\wedge R)$

(4) **分配律** $P\vee(Q\wedge R)\Leftrightarrow(P\vee Q)\wedge(P\vee R)$

$P\wedge(Q\vee R)\Leftrightarrow(P\wedge Q)\vee(P\wedge R)$

(5) **（德）摩根定律** $\neg(P\vee Q)\Leftrightarrow\neg P\wedge\neg Q$，$\neg(P\wedge Q)\Leftrightarrow\neg P\vee\neg Q$

(6) **吸收律** $P\vee(P\wedge Q)\Leftrightarrow P$，$P\wedge(P\vee Q)\Leftrightarrow P$

(7) **补余律** $P\vee\neg P\Leftrightarrow T$，$P\wedge\neg P\Leftrightarrow F$

(8) **连词化归律** $P\rightarrow Q\Leftrightarrow\neg P\vee Q$

$P\leftrightarrow Q\Leftrightarrow(P\rightarrow Q)\wedge(Q\rightarrow P)$

$P\leftrightarrow Q\Leftrightarrow(P\wedge Q)\vee(\neg Q\wedge\neg P)$

(9) **量词转换律** $\neg(\exists x)P\Leftrightarrow(\forall x)(\neg P)$，$\neg(\forall x)P\Leftrightarrow(\exists x)(\neg P)$

(10) **量词分配律** $(\forall x)(P\wedge Q)\Leftrightarrow(\forall x)P\wedge(\forall x)Q$

$(\exists x)(P\vee Q)\Leftrightarrow(\exists x)P\vee(\exists x)Q$

永真蕴含式：

> 化简式：
> $P \wedge Q \Rightarrow P$
> $P \wedge Q \Rightarrow Q$
>
> 附加式
> $P \Rightarrow P \vee Q$
> $Q \Rightarrow P \vee Q$
>
> 析取三段论
> $\neg P, P \vee Q \Rightarrow Q$
>
> 假言推理
> $P, P \rightarrow Q \Rightarrow Q$
>
> 拒取式
> $\neg Q, P \rightarrow Q \Rightarrow \neg P$
>
> 假言三段论
> $P \rightarrow Q, Q \rightarrow R \Rightarrow P \rightarrow R$
>
> 二难推理
> $P \vee Q, P \rightarrow R, Q \rightarrow R \Rightarrow R$
>
> 全称固化 【y是个体域中任一个体，消去全称量词】
> $(\forall x) P(x) \Rightarrow P(y)$
>
> 存在固化 【y是某个是P(y)为真的个体，消去存在量词】
> $(\exists x) P(x) \Rightarrow P(y)$

自然演绎推理

是从一组一直为真的事实出发，直接运用经典逻辑的推理规则推出结论的过程。

常用推理规则

- P规则：在推理的任何步骤都可引入前提
- T规则：推理时，如果前面步骤中有一个或多个公式永真蕴含公式S，则可把S引入推理过程中
- 反证法：$P \Rightarrow Q$，当且仅当$P \wedge \neg Q \Leftrightarrow F$
- 假言推理

$$\neg P, P \rightarrow Q \Rightarrow Q$$

- 拒取式推理

$$\neg\neg Q, P \rightarrow Q \Rightarrow \neg P$$

- 假言三段论

$$P \rightarrow Q, Q \rightarrow R \Rightarrow P \rightarrow R$$

5.归结演绎推理

鲁滨逊归结原理（消解原理）：把永真性的证明转化为不可满足性的证明【反证法】

合取范式：

设 $A=B_1 \wedge B_2 \wedge \ldots \wedge B_n$

其中，$B_i=L_1 \vee L_2 \vee \ldots \vee L_{mi}$，而$L_j$为原子公式或其否定。则称A为合取范式。

如：$P(x) \wedge (P(x) \vee Q(y) \vee \neg R(x,y))$

析取范式：

设 $A=B_1 \vee B_2 \vee \ldots \vee B_n$

其中，$B_i=L_1 \wedge L_2 \wedge \ldots \wedge L_{mi}$，而$L_j$为原子公式或其否定。则称A为析取范式。

如：$P(x) \vee (P(x) \wedge Q(y) \wedge \neg R(x,y))$

前束型范式：

- 一个谓词公式的所有量词均非否定地出现在公式的最前面，且它的辖域一直延伸到公式之末，同时公式中不出现连接词→及↔。
  - 例：$(\forall x)(\exists y)(\forall z)(P(x) \wedge F(y,z) \wedge Q(y,z))$
- 任一谓词公式均可化为与其对应的前束范式，其化简方法将在后面子句集的化简中讨论。

斯克林范式（Skolem）：

- 在前束范式的首标中不出现存在量词，即从前束范式中消去全部存在量词所得的公式。
- 其一般形式为：
- $(\forall x_1)(\forall x_2)\cdots(\forall x_3)M(x_1, x_2, \cdots x_3)$
- 其中$M(x_1, x_2, \cdots x_3)$是一个合取范式，称为Skolem标准型的母式

- 用f(x)代替上面前束范式中的y即得到Skolem范式
- 其中f(x)称为Skolem函数。其一般式为：

  $f(x_1,x_2,\ldots,x_n)$

子句

> 文字：原子谓词公式及其否定
>
> 子句：文字的析取式
>
> 空子句：永假，不含任何文字
>
> 子句集：子句或空子句构成的集合

化简子句集(9个步骤)

1. 消去连接词 → 和 ↔（等价公式）
2. 减少否定符号的辖域（利用等价关系把“¬”移到紧靠谓词的位置上）
3. 变元标准化（不同名字）
4. 化为前束范式

5. 消去存在量词
    - 其左边没有全称量词，新的个体常量替换即可
    - 左边有全称量词，要用skolem函数替换
6. 化为skolem标准型
7. 消去全称量词
8. 消去合取词
9. 不同子句的变元不同名

鲁滨逊归结原理

欲证明问题的否定加入子句集。希望导出空子句或不能继续归结为止。

- 命题逻辑归结原理：直接消去互补文字
- 谓词逻辑归结原理：置换、合一

置换

置换

– 置换是一个形如{ $t_1/x_1, t_2/x_2, \cdots, t_n/x_n$ }的有限集合：

– t_i是项，x_i是变元，t_i/x_i表示用t_i替换x_i。

– $t_i \neq x_i$, $x_i \neq x_j (i \neq j), i,j=1,2,3\cdots,n$

– x_i不可循环出现在t_j中。

– 不含任何元素的置换称为空置换，以ε表示

例如：

{ a/x, f(b)/y, w/z }、{ g(a)/x, f(b)/y }

是一个置换，但：

{ g(y)/x, f(x)/y }

不是置换。

置换乘法（两个置换的合成）： $\theta \cdot \lambda$

– 设有2个替换：

– $\theta = \{ t_1/x_1, t_2/x_2, \cdots, t_n/x_n \}$

– $\lambda = \{ u_1/y_1, u_2/y_2, \cdots, u_m/y_m \}$

– 令：$\mu = \{ t_1 \cdot \lambda/x_1, t_2 \cdot \lambda/x_2, \cdots, t_n \cdot \lambda/x_n , u_1/y_1, u_2/y_2, \cdots, u_m/y_m \}$

– 从上述集合中删去下面两种元素：

（1）当$t_i \cdot \lambda = x_i$时，从μ中删除$t_i \cdot \lambda/x_i$

（2）当$y_i = x_j$时，从μ中删除u_i/y_i

– 剩余元素所构成的集合仍然是一个置换，称为θ与λ的乘积，记作：$\theta \cdot \lambda$

- 后者如果能够替换前者的分子，就替换（前者分子含有后者分母元素）
- 后者分母如果与前者分母相同，删除后者
- 分子分母相同，则删去

例1、设有置换：

$\theta = \{ f(y)/x, z/y \}$ ， $\lambda = \{ a/x, b/y, y/z \}$

$\theta \cdot \lambda = \{ f(b)/x, y/z \}$

合一

一个置换能将集合中所有表达式化成一样的形式。这叫合一置换

合一

- 设有公式集：F={ F1, F2, …, Fn }，若存在一个置换θ，使得
- F1θ = F2θ = … =Fnθ
- 则称θ为F的一个合一置换，且称F1, F2, …, Fn是可合一的。

{P(x,f(y),B),P(z,f(B),B)}的合一有：

- s={A/x,B/y,A/z}
- s={z/x,B/y}
- s={x/z,B/y}
- 它们都是这两个谓词公式的合一者。

结论：合一者不唯一。

- 最一般合一(mgu)
  - 设σ是公式集F的一个合一，如果对于F的任何一个合一θ，都存在置换λ，使得：θ=σ·λ
  - 则称σ是F的最一般合一。
- 例：

E1=P(y),E2=P(f(z))是可合一的，其合一置换是θ ={f(a)/y,a/z}

但E1和E2的mgu是{f(z)/y}。

最一般合一者

- 置换最少，限制最小，产生的置换结果最具一般性。

例：

- E1=Q(y),E2=Q(z)
- 对于E1和E2来说，σ ={y/z}和σ ={z/y}都是它的mgu

结论：mgu也不是唯一的。

差异集

- 设F是一个非空原子公式集，且各公式的谓词名相同，依次比较对应项，若某项不完全相同，则由它们不相同的部分组成的集合称为一个差异集。

- 每项对应比较，不相同的就是差异集

F={ P(x,y,z), P(x,f(a),h(b)) }

差异集为：D1={ y, f(a) }, D2={ z, h(b) }

求最一般合一

（1）令 k=0, $F_k=F$, $\sigma_k=\varepsilon$（空置换，表示不做置换）
（2）若F_k只含一个公式，则σ_k即为所求，否则：
（3）求F_k的差异集D_k
（4）若D_k中存在元素x_k和t_k，其中x_k是变元，t_k是项，且x_k不在t_k中出现，则令：

$\sigma_{k+1}=\sigma_k \cdot \{t_k/x_k\}$
$F_{k+1}=F_k\{t_k/x_k\}$
$K=k+1$

转（2）

18

（5）否则，F的最一般合一不存在。

不断求F中公示的差异集，并置换公式，直到只含有一个公式时，最后一步的置换就是最一般合一

求最一般合一： $F=\{P(a, x, f(g(y))), P(z, f(z), f(u))\}$

令 k=0, $F_0=F$, $\sigma_0=\varepsilon$，因F_0中含有两个表达式，
故σ_0不是解
有差异集：$D_0=\{a, z\}$
$\sigma_1=\sigma_0 \cdot \{a/z\}=\{a/z\}$
$F_1=\{P(a, x, f(g(y))), P(z, f(z), f(u))\}\{a/z\}$
$=\{P(a, x, f(g(y))), P(a, f(a), f(u))\}$
$D1=\{x, f(a)\}$
$\sigma 2=\sigma 1 \cdot \{f(a)/x\}=\{a/z, f(a)/x\}$
$F2=\{P(a, x,f(g(y))), P(a, f(a), f(u))\}\{f(a)/x\}$
$=\{P(a, f(a), f(g(y))), P(a,f(a),f(u))\}$
$D2=\{g(y), u\}$
$\sigma 3=\sigma 2 \cdot \{g(y)/u\}=\{a/z, f(a)/x, g(y)/u\}$
$F3=\{P(a,f(a), f(g(y))), P(a,f(a),f(u))\}\{g(y)/u\}$
$=\{P(a, f(a), f(g(y))), P(a,f(a),f(g(y)))\}$
$=\{P(a, f(a), f(g(y)))\}$
因为F3中只含有一个表达式，故其最一般合一是：$\sigma_3=\{a/z, f(a)/x, g(y)/u\}$

合一定理：任何一个可合一的非空有限集，一定存在最一般合一

二元归结式

- 设C_1、C_2是两个无公共变元的子句，且L_1、L_2分别是C_1、C_2中的文字，若L_1与$\neg L_2$存在最一般合一σ，则称

$$C_{12}=\{C_1\sigma-\{L_1\sigma\}\} \cup \{C_2\sigma-\{L_2\sigma\}\}$$

为C_1与C_2的二元归结式，C_1、C_2为C_{12}的亲本子句，L_1、L_2称为消解文字（归结式上的文字）。

例 设$C_1=P(x) \vee Q(a)$，$C_2=\neg P(b) \vee R(x)$，求C_{12}

解：

由于C_1和C_2有相同的变元x，不符合谓词逻辑归结定义的要求。为了进行归结，需要修改C_2中变元x的名字，令

$C_2=\neg P(b) \vee R(y)$

此时$L_1=P(x)$, $L_2=\neg P(b)$，L_1和L_2的合一是$\sigma=\{b/x\}$。

则有

$C_{12}=(\{C_1\sigma\}-\{L_1\sigma\}) \cup (\{C_2\sigma\}-\{L_2\sigma\})$
$=(\{P(b), Q(a)\}-\{P(b)\}) \cup (\{\neg P(b), R(y)\}-\{\neg P(b)\})$
$=(\{Q(a)\}) \cup (\{R(y)\})=\{Q(a), R(y)\}$
$=Q(a) \vee R(y)$

求归结式不能同时消去两个互补对，其结果不是二元归结式。

注意

- 如果C_1,C_2中有相同的变元，需要修改其中一个的变元名字。

  例：$c_1=P(x)\vee Q(a)$,$c_2=\neg P(b)\vee R(x)$

- 如果参加归结的子句内部有可合一的文字，则在归结前对这些文字先进行合一。

  例：$c1=P(x)\vee P(f(a))\vee Q(x)$

  $c2=\neg P(y)\vee R(b)$

定理证明

步骤：

- 定义谓词
- 谓词公式表示结论和前提。结论用否定表示
- 化为子句集
- 归结为NIL即证明

问题求解

步骤：

- 谓词公式表示已知条件，化为子句集 S
- 所求问题用谓词公式 P' 表示，引入ANSWER，令

  $P=\neg P'\vee ANSWER$

  ANSWER的变元必须与P’中的变元相同。
- P 化为子句集并入 S，得到 S' 并归结
- 得到归结式ANSWER，则答案在answer中

归结演绎推理的方法

命题逻辑的归结演绎推理

归结原理

假设F为已知前提，G为欲证明的结论，归结原理把证明G为F的逻辑结论转化为证明$F\wedge\neg G$为不可满足。

再根据定理3.1，在不可满足的意义上，公式集$F\wedge\neg G$与其子句集是等价的，即把公式集上的不可满足转化为子句集上的不可满足。

归结反演

应用归结原理证明定理的过程称为归结反演。

归结反演过程

在命题逻辑中，已知F，证明G为真的归结反演过程如下：

①否定目标公式G，得$\neg G$;

②把$\neg G$并入到公式集F中，得到$\{F，\neg G\}$；

③把$\{F，\neg G\}$化为子句集S。

④ 应用归结原理对子句集S中的子句进行归结，并把每次得到的归结式并入S中。如此反复进行，若出现空子句，则停止归结，此时就证明了G为真。

命题逻辑的归结演绎推理(2/2)

例 设已知的公式集为{P, (P∧Q)→R, (S∨T)→Q, T}，求证结论R

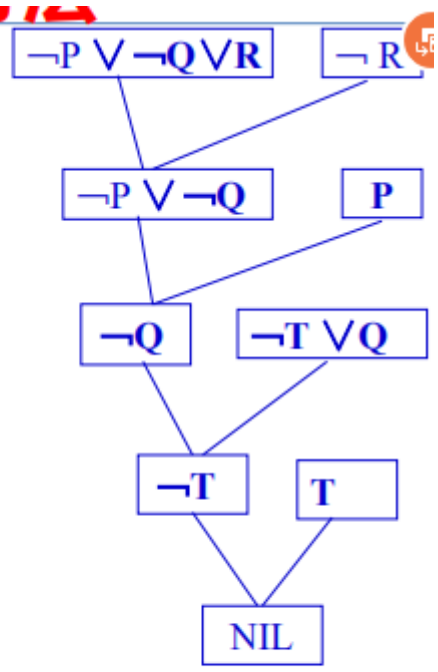
解：假设结论R为假, 将¬R加入公式集，并化为子句集

S={P,¬P∨¬Q∨R, ¬S∨Q, ¬T∨Q, T, ¬R}

其归结过程如右图的归结树所示。

其含义为：先假设子句集S中的所有子句均为真，即原公式集为真,¬R也为真；然后利用归结原理，对子句集进行归结，并把所得的归结式并入子句集中；重复这一过程，最后归结出了空子句。

根据归结原理的完备性，可知子句集S是不可满足的，即开始时假设¬R为真是



谓词逻辑的归结演绎推理

谓词逻辑的归结反演

谓词逻辑的归结演绎推过程与命题逻辑的归结演绎推理过程相比，其步骤基本相同，但每步的处理对象不同。例如，在步骤(3)化简子句集时，谓词逻辑需要把由谓词构成的公式集化为子句集；在步骤(4)按归结原理进行归结时，谓词逻辑的归结原理需要考虑两个亲本子句的合一。

例2.30已知

F: (∀x)((∃y)(A(x, y)∧B(y))→(∃y)(C(y)∧D(x, y)))

G: ¬(∃x)C(x)→(∀x)(∀y)(A(x, y)→¬B(y))

求证G是F的逻辑结论。

证明：先把G否定，并放入F中，得到的{F, ¬G}为

{(∀x)((∃y)(A(x,y)∧B(y))→(∃y)(C(y)∧D(x,y))),

¬(¬(∃x)C(x)→(∀x)(∀y)(A(x,y)→¬B(y)))}

$\neg\neg P \Leftrightarrow P$

$P \vee Q \Leftrightarrow Q \vee P,\ P \wedge Q \Leftrightarrow Q \wedge P$

$(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$

$(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$

$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$

$P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$

$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$

$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$

$P \vee (P \wedge Q) \Leftrightarrow P,\ P \wedge (P \vee Q) \Leftrightarrow P$

$P \vee \neg P \Leftrightarrow T,\ P \wedge \neg P \Leftrightarrow F$

$P \rightarrow Q \Leftrightarrow \neg P \vee Q$

$P \leftrightarrow Q \Leftrightarrow (P \rightarrow Q) \wedge (Q \rightarrow P)$

$P \leftrightarrow Q \Leftrightarrow (P \wedge Q) \vee (\neg Q \wedge \neg P)$

$\neg(\exists x)P(x) \Leftrightarrow (\forall x)(\neg P(x))$

$\neg(\forall x)P(x) \Leftrightarrow (\exists x)(\neg P(x))$

$(\forall x)(P(x) \wedge Q(x)) \Leftrightarrow (\forall x)P(x) \wedge (\forall x)Q(x)$

$(\exists x)(P(x) \vee Q(x)) \Leftrightarrow (\exists x)P(x) \vee (\exists x)Q(x)$

再把{F,¬G}化成子句集，得到

(1) ¬A(x,y)∨¬B(y)∨C(f(x))

(2) ¬A(u,v)∨¬B(v)∨D(u,f(u))

(3) ¬C(z)

(4) A(m,n)

(5) B(k)

其中，(1)、(2)是由F 化出的两个子句，(3)、(4)、(5)是由¬G化出的3个子句。

最后应用谓词逻辑的归结原理对上述子句集进行归结，其过程为

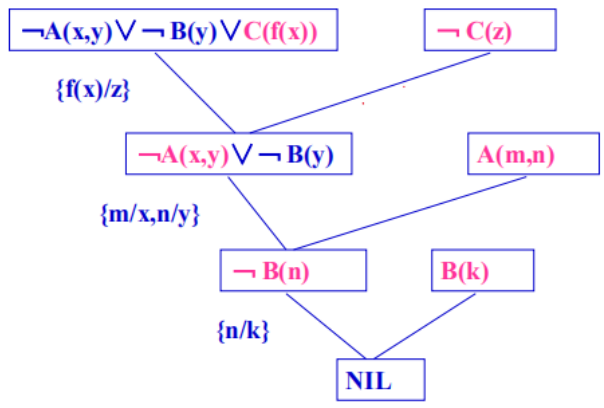
(6) ¬A(x,y)∨¬B(y)　由(1)和(3)归结，取σ={f(x)/z}

(7) ¬B(n)　由(4)和(6)归结，取σ={m/x,n/y}

(8) NIL　由(5)和(7)归结，取σ={n/k}

因此，G是F 的逻辑结论。

上述归结过程可用如下归结树来表示



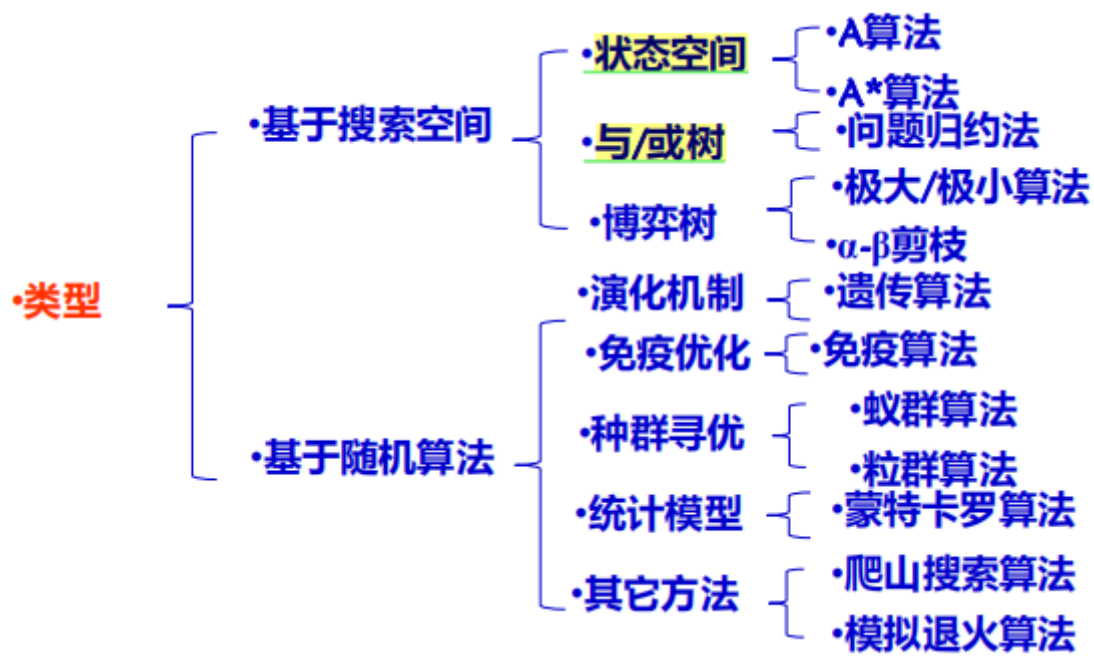
主观贝叶斯?

9.状态空间

搜索概述

搜索：依靠经验，利用已有知识，根据问题的实际情况，不断寻找可利用知识，从而构造一条代价最小的推理路线，使问题得以解决的过程

搜索类型



盲目搜索

搜索之前就预定好控制策略，整个搜索过程中的策略不再改变。灵活性，效率低。

bfs、dfs

智能搜索

是指可以利用搜索过程得到的中间信息来引导搜索向最优方向发展的算法。

状态空间法

- 状态：表示问题求解过程中每一步问题状况的数据结构
- 操作、算符：把问题从一种状态变换为另一种状态的手段
- 状态空间方法：基于解答空间的问题表示和求解方法，它是以状态和操作（算符）为基础来表示和求解问题的。
- 问题的状态空间：是一个表示该问题全部可能状态、可用算符的集合，它包含三种说明的集合，即三元状态（S，F，G）。**S为问题的所有初始状态集合、F为操作的集合、G为目标状态的集合。**
- 问题的解：从问题的初始状态集，经过一系统列的算符运算，到达目标状态，所经过算符的序列构成一个问题的解

10.问题规约

- **问题归约法**

  – 对问题进行分解或变换

  – 当问题比较复杂，通过分解或变换，将其转换为一系列较为简单的问题。

  – 通过对这些较简单的问题的求解来实现对原问题的求解

分解

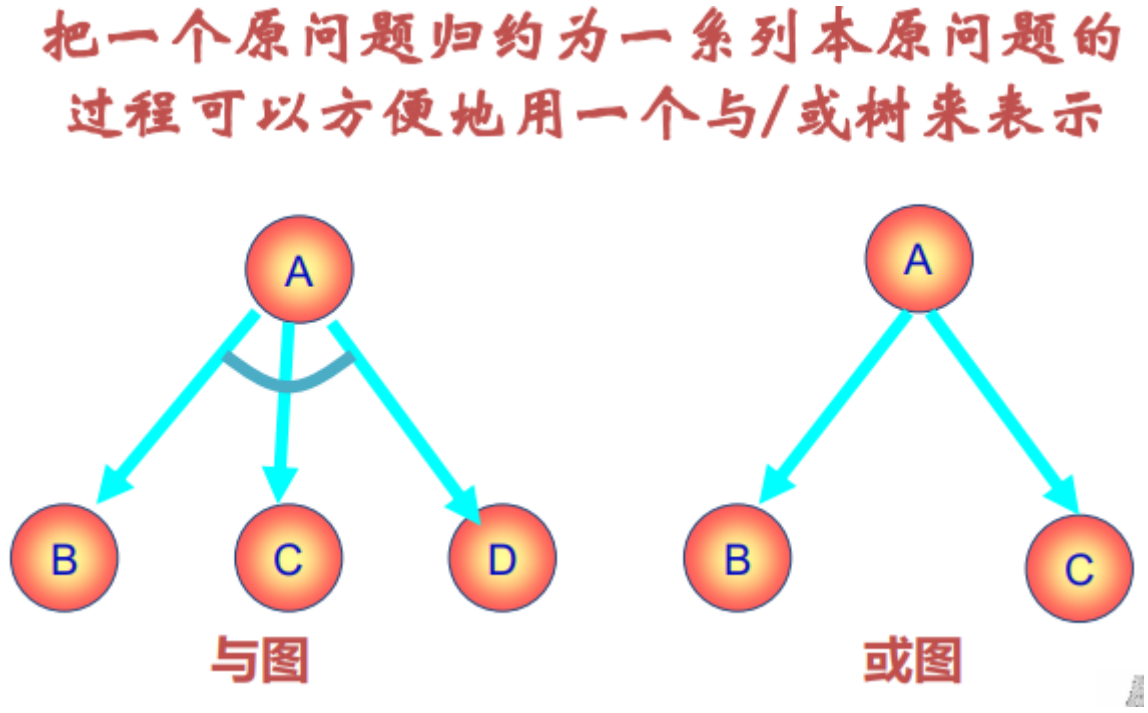
> - 如果一个问题P可以归约为一组子问题P1,P2,...,Pn，并且只有当所有子问题Pi都有解时原问题P才有解，任何一个子问题Pi无解都会导致原问题P无解，则称此种归约为问题的分解。
> - 即分解所得到的子问题的“与”与原问
>
> 题P等价。

等价变换

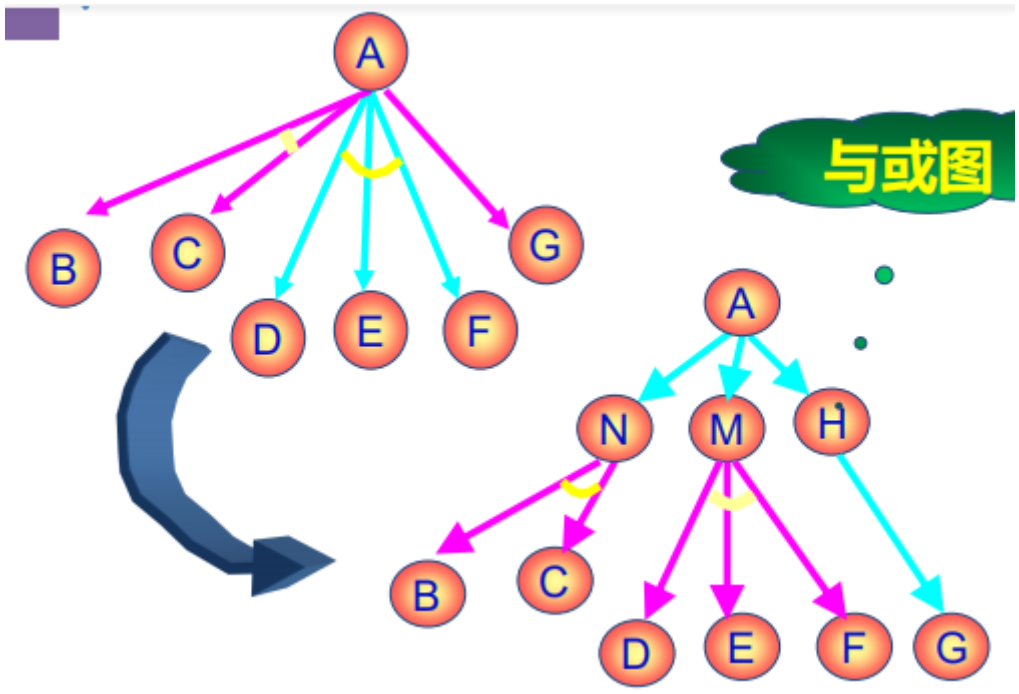
> - 如果一个问题P可以归约为一组子问题P1,P2,...,Pn，并且子问题Pi中只要有一个有解则原问题P就有解，只有当所有子问题Pi都无解时原问题P才无解，称此种归约为问题的等价变换，简称变换。
> - 即变换所得到的子问题的“或”与原问题P等价。

问题归约 - 把一个复杂问题分解或变换为一组本原问题的过程称作归约。

本原问题– 是指那种不能（或不需要）再进行分解或变换，且可以直接解答的子问题。



与或图



“与节点”与“或节点”

–如果一个节点的所有子节点是与关系，则称该节点为与节点，如果其所有子节点是或关系，则称该节点为或节点。

端节点与终止节点

–在与或树中没有子节点的节点称为**端节点**

–本原问题所对应的节点称为**终止节点**。

–显然，终止节点一定是端节点，但端节点**不一定是终止节点。

有解节点&不可解节点

在与/或树中，满足以下三个条件之一的节点为可解节点：

①任何终止节点都是可解节点。

②对“或”节点，当其子节点中至少有一个为可解节点时，则该或节点就是可解节点。

③对“与”节点，只有当其子节点全部为可解节点时，该与节点才是可解节点。

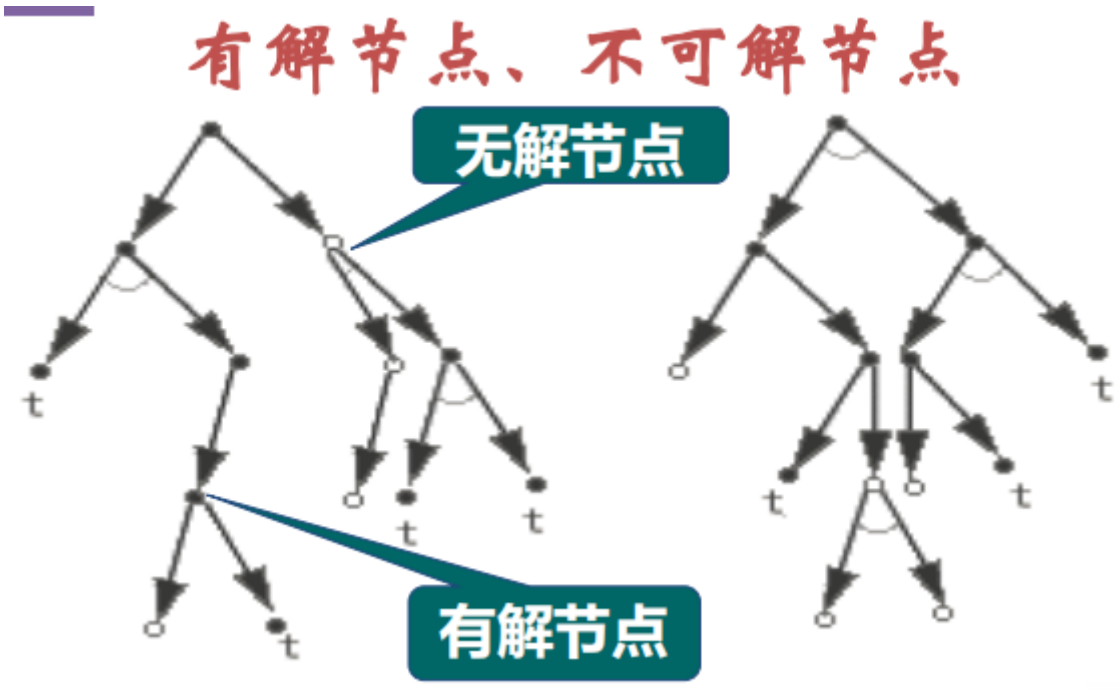
同样，可用类似的方法定义不可解节点：

①不为终止节点的端节点是不可解节点。

②对“或”节点，若其全部子节点都为不可解节点，则该或节点是不可解节点。

③对“与”节点，只要其子节点中有一个为不可解节点，则该与节点是不可解节点。

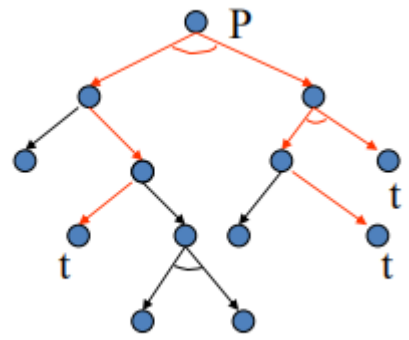
举例



解树

- 由可解节点构成，并且由这些可解节点可以推出初始节点（它对应着原始问题）为可解节点的子树为解树。
- 在解树中一定包含初始节点。

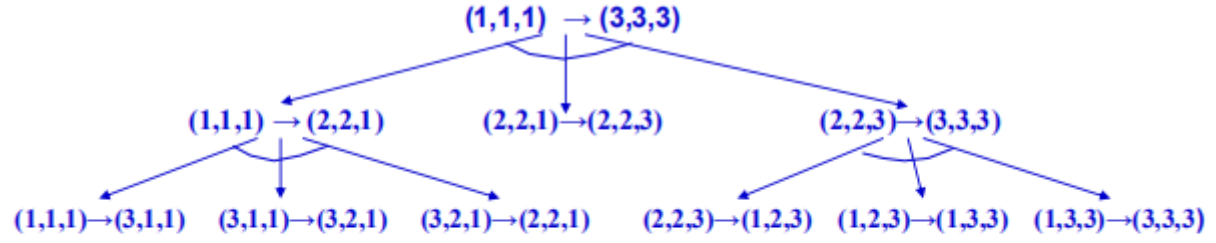
- 例如，
- 右图给出的与或树中，用红线表示的子树是一个解树。在该图中，节点P为原始问题节点，用t标出的节点是终止节点。根据可解节点的定义，很容易推出原始问题P为可解节点。



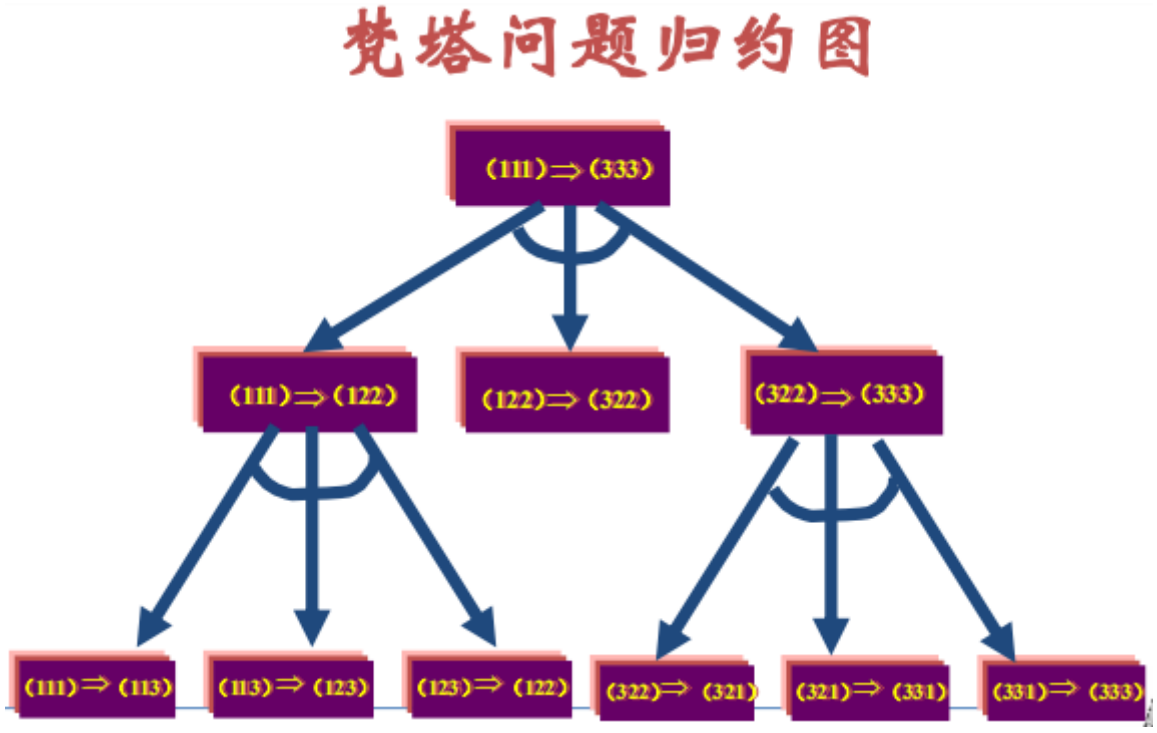
> 问题归约求解过程就实际上就是生成解树，即证明原始节点是可解节点的过程。这一过程涉及到搜索的问题。

三阶樊塔问题

利用问题归约方法，原问题可分解为以下三个子问题：
(1) 把金片A及B移到2号钢针上的双金片移动问题。即(1, 1, 1)→(2, 2, 1)
(2) 把金片C移到3号钢针上的单金片移动问题。即 (2, 2, 1)→(2, 2, 3)
(3) 把金片A及B移到3号钢针的双金片移动问题。即(2, 2, 3)→((3, 3, 3)
其中，子问题(1)和(3)都是一个二阶梵塔问题，它们都还可以再继续进行分解；子问题(2)是本原问题，它已不需要再分解。
三阶梵塔问题的分解过程可用如下图与/或树来表示



在该与/或树中，有7个终止节点，它们分别对应着7个本原问题。如果把这些本原问题从左至右排列起来，即得到了原始问题的解：
(1, 1, 1)→(3, 1, 1) (3, 1, 1)→(3, 2, 1) (3, 2, 1)→(2, 2, 1) (2, 2, 1)→(2, 2, 3)
(2, 2, 3)→(1, 2, 3) (1, 2, 3)→(1, 3, 3) (1, 3, 3)→(3, 3, 3)



11.启发式搜索

不一定能找到最优解，但相对盲目搜索更快

- **启发信息**

  用于指导搜索过程且与具体问题求解有关的控制信息

- **启发信息分类**

  – 有效地帮助确定扩展节点的信息。

  – 在扩展节点时，用于决定要生成哪一个或哪几个后继节点 。

  – 用于确定某些应该从搜索树中抛弃或修剪的节点。

- **估价函数**

在扩展节点时，用来描述节点重要程度的函数称为估价函数。其一般形式为：

$$f(x)=g(x)+h(x)$$

其中，

g(x)：为初始节点S0到节点x已实际付出的代价，

h(x)：是从节点x到目标节点Sg的最优路径的估计代价，

启发信息主要由h(x)来体现，故把它称为启发函数。

A算法

- 两种数据结构：
  - Open表：存放未扩展的节点
  - Closed表：存放已扩展的节点
- 在状态空间搜索中，如果每一步都利用估价函数f(n)=g(n)+h(n)对Open表中的节点进行排序，则称A算法。它是一种为启发式搜索算法。

排序方式

- 全局择优： 从Open表的所有节点中选择一个估价函数值最小的进行扩展。
- 局部择优：仅从刚生成的子节点中选择一个估价函数值最小的进行扩展。

全局择优

(1)把初始节点S_0放入Open表中，$f(S_0)=g(S_0)+h(S_0)$；
(2)如果Open表为空，则问题无解，失败退出；
(3)把Open表的第一个节点取出放入Closed表，并记该节点为n；
(4)考察节点n是否为目标节点。若是，则找到了问题的解，成功退出；
(5)若节点n不可扩展，则转第(2)步；
(6)扩展节点n，生成其子节点$n_i(i=1,2,\cdots)$，计算每一个子节点的估价值$f(n_i)(i=1,2,\cdots)$，并为每一个子节点设置指向父节点的指针，然后将这些子节点放入Open表中；
(7)根据各节点的估价函数值，对Open表中的全部节点按从小到大的顺序重新进行排序；
(8)转第(2)步。

局部择优

将初始节点S0放入OPEN表中，计算估价函数f(s0)。
若OPEN表为空，则问题无解，退出。
将OPEN表中第1个节点N放入CLOSED表中，并按顺序编号为n。
若$N=S_g$，则已求得问题的解，退出。
生成N的所有子节点集合SUB，并且删除已在OPEN及CLOSED中存在的节点。
计算SUB中每个节点i的估价函数值f(i)，并对SUB中的所有节点按照估价值进行升序排序
将SUB中的所有节点放入OPEN的首部，并填入其父节点编号n，转（2）。

A*算法

假设$f^*(n)$为从初始节点S_0出发，约束经过节点n到达目标节点S_g的最小代价值。估价函数$f(n)$则是$f^*(n)$的估计值。显然，$f^*(n)$应由以下两部分所组成：一部分是从初始节点S_0到节点n的最小代价，记为$g^*(n)$；另一部分是从节点n到目标节点S_g的最小代价，记为$h^*(n)$，当问题有多个目标节点时，应取其中代价最小的一个。因此有

$$f^*(n)=g^*(n)+h^*(n)$$

把估价函数$f(n)$与$f^*(n)$相比，$g(n)$是对$g^*(n)$的一个估计，$h(n)$是对$h^*(n)$的一个估计。在这两个估计中，尽管$g(n)$的值容易计算，但它不一定是从初始节点S_0到节点n的真正最小代价，很有可能从初始节点S_0到节点n的真正最小代价还没有找到，故$g(n)\geq g^*(n)$。

有了$g^*(n)$和$h^*(n)$的定义，如果对A算法（全局择优的启发式搜索算法）中的$g(n)$和$h(n)$分别提出如下限制：

第一，$g(n)$是对$g^*(n)$的估计，且$g(n)>0$；

第二，$h(n)$是$h^*(n)$的下界，即对任意节点n均有$h(n)\leq h^*(n)$。

则称得到的算法为A^*算法。

可采纳性的含义：

对任一状态空间图，当从初始节点到目标节点有路经存在时，如果搜索算法总能在有限步骤内找到一条从初始节点到目标节点的最佳路径，并在此路径上结束，则称该搜索算法是可采纳的。

定理4.1 对有限图，如果从初始节点S_0到目标节点S_g有路径存在，则算法A*一定成功结束。

引理4.2 在A*算法终止前的任何时刻，Open表中总存在节点n'，它是从初始节点S_0到目标节点的最佳路径上的一个节点，且满足$f(n')\leq f^*(S_0)$。

定理4.2 对无限图，若从初始节点S_0到目标节点S_g有路径存在，则A*算法必然会结束。

推论4.1 Open表中任一具有$f(n)<f^*(S_0)$的节点n，最终都被A*算法选作为扩展的节点。

定理4.3 A*算法是可采纳的，即若存在从初始节点S_0到目标节点S_g的路径，则A*算法必能结束在最佳路径上。

推论4.2 在A*算法中，对任何被扩展的节点n，都有$f(n)\leq f^*(S_0)$。

定理4.4

设有两个A*算法A_1*和A_2*，它们有

A_1*：$f_1(n)=g_1(n)+h_1(n)$

A_2*：$f_2(n)=g_2(n)+h_2(n)$

如果A_2*比A_1*有更多的启发性信息，即对所有非目标节点均有

$h_2(n)>h_1(n)$

则在搜索过程中，被A_2*扩展的节点也必然被A_1*扩展，即A_1*扩展的节点不会比A_2*扩展的节点少，亦即A_2*扩展的节点集是A_1*扩展的节点集的子集。

A*算法的单调性

在A*算法中，每当扩展一个节点n时，都需要检查其子节点是否已在Open表或Closed表中。

对已在Open表中的子节点，需要决定是否调整指向其父节点的指针；

对已在Closed表中的子节点，除需要决定是否调整其指向父节点的指针外，还需要决定是否调整其子节点的后继节点的父指针。

如果能够保证，每当扩展一个节点时就已经找到了通往这个节点的最佳路径，就没有必要再去作上述检查

为满足这一要求，我们需要对启发函数h(n)增加单调性限制。

定义4.1 如果启发函数满足以下两个条件:

(1) $h(S_g)=0$;

(2) 对任意节点n_i及其任一子节点n_j，都有

$$0 \le h(n_i)-h(n_j) \le c(n_i, n_j)$$

其中$c(n_i, n_j)$是n_i到其子节点n_j的边代价，则称$h(n)$满足单调限制。

也可以写成:

$$h(n_i) \le h(n_j)+ c(n_i, n_j)$$

这说明，从节点n_i到目标节点最小代价的估值不会超过从节点n_i到其子节点n_j的边代价加上从n_j到目标节点的最小代价估值。

定理4.5 如果h满足单调条件，则当A*算法扩展节点n时，该节点就已经找到了通往它的最佳路径，即$g(n)=g^*(n)$。

定理4.6 如果$h(n)$满足单调限制，则A*算法扩展的节点序列的f 值是非递减的，即$f(n_i) \le f(n_{i+1})$。

A*算法的最优性

- A*算法的搜索效率很大程度上取决于启发函数h(n)。一般来说，在满足$h(n) \leqslant h^*(n)$的前提下，h(n)的值越大越好。h(n)的值越大，说明它携带的启发性信息越多，A*算法搜索时扩展的节点就越少，搜索效率就越高。
- A*算法的这一特性被称为最优性。

12.与或树的搜索

一般搜索思想

- 可解标示过程

  –由可解节点来确定父节点、祖父节点等为可解节点的过程

- 不可解标示过程

  –由不可解节点来确定父节点、祖父节点等为不可解节点的过程

一般搜索过程

- 把原始问题作为初始节点S_0，并把它作为当前节点
- 应用分解或等价变换算符对当前节点进行扩展
- 为每个子节点设置指向父节点的指针
- 选择合适的节点作为当前节点，反复执行前两步，并多次调用可解标示过程和不可解标示过程，直到初始节点被标示为可解或不可解节点。
- 标示可解与不可解的过程是自下而上进行的。
- 如果已确定某个节点为可解节点，则其不可解的后继节点可以从搜索树中删除。
- 如果确定某个节点是不可解节点，则其全部后继节点都可以从搜索树中删除。

与或树的广搜

① 将初始节点S_0放入OPEN表中。

② 将OPEN表中第1个节点N放入CLOSED表中，并按顺序编号为n。

③ 若N有子节点存在，则执行下述操作：

- 生成N的所有子节点，将其放入OPEN的尾部，并填入其父节点编号n。
- 考察这些子节点中是否有终止节点，若有，则标记这些终止节点为可解节点，并将其放入CLOSED表中，然后由它们的可解性返回去推断其先辈的可解性，并对可解节点进行标记，若S0为可解节点，则搜索成功，退出。
- 从OPEN表中删除其先辈为可解的所有节点，转（2）。

④ 若N不可扩展，则执行下述操作：

- 判断N是否为不可解节点，
- 由N的不可解性返回去推断其先辈的不可解性，并对不可解节点进行标记，若S0为不可解节点，则搜索失败，退出。
- 从OPEN表中删除其先辈为不可解的所有节点，转（2）。
- 若为可解节点，则同(3)子项上操作

与或树的深搜

① 将初始节点S_0放入OPEN表中。

② 将OPEN表中第1个节点N放入CLOSED表中，并按顺序编号为n。

③ 若N有子节点存在，则执行下述操作：

- 生成N的所有子节点，将其放入OPEN的首部，并填入其父节点编号n。

- 考察这些子节点中是否有终止节点，若有，则标记这些终止节点为可解节点，并将其放入CLOSED表中，然后由它们的可解性返回去推断其先辈的可解性，并对可解节点进行标记，若S0为可解节点，则搜索成功，退出。
- 从OPEN表中删除其先辈为可解的所有节点，转（2）。

④ 若N无子节点存在，则执行下述操作：

- 判断N是否为不可解节点，
- 由N的不可解性返回去推断其先辈的不可解性，并对不可解节点进行标记，若S0为不可解节点，则搜索失败，退出。
- 从OPEN表中删除其先辈为不可解的所有节点，转（2）。
- 若为可解节点，则同(3)子项上操作

解树的代价

设g(x)表示节点x的代价，c(x,y)表示节点x到其子节点y的代价。

(1)若x是终止节点，则g(x)=0

(2)若x是或节点，其子节点依次为$y_1,y_2,\cdots,y_n$，则有：

$$g(x)=\min_{1\leqslant i\leqslant n}\{c(x,y_i)+g(y_i)\}$$

(3)若x是与节点，其子节点依次为$y_1,y_2,\cdots,y_n$，则有：

和代价法：$g(x)=\sum(c(x,yi)+g(yi))$

最大代价法：$g(x)=\max_{1\leqslant i\leqslant n}\{c(x,y_i)+g(y_i)\}$

(4)若x不可扩展，则：$g(x)=\infty$

若问题有解的话，则子节点的代价可以推算出父节点的代价，直到初始节点的代价，S0的代价就是解树的代价。

希望树

为了找到最优解，搜索过程的任何时刻都应该选择最有希望成为最优解树的一部分的节点进行扩展。由于这些节点及其父节点所构成的与或树最有可以能成为最优解树的一部分，因此称它为希望解树，简称希望树。注意，希望树是会随搜索过程而不断变化的。

- 初始节点S0在希望树中。
- 如果节点x在希望树中，则：
  - 如果x是或节点，且其子节点依次为$y_1,y_2,\cdots,y_n$，则具有下述值的子节点也在希望树中：

$$\min\{c(x,y_i)+g(y_i)\}$$

$$1\leqslant i\leqslant n$$

  - 若x是与节点，则其所有子节点都在希望树中。

13.博弈树

概念

- “二人零和、全信息、非偶然”博弈的特点
  - 博弈A、B双方轮流行动，博弈结果只有A胜、A负、平局三种情况。
  - 博弈双方都了解当前和格局及过去的历史。
  - 任何一方在采取行动前都要根据当前的实际情况进行得失分析，选取自己最有利而对对方最不利的对策，不存在“碰运气”的偶然因素。

博弈树

- 在博弈过程中，己方的各种攻击方案为“或”关系，而对方的应着方案为“与”关系。描述博弈过程的“与\或”树称为博弈树。

注：“与\或”树始终是站在某一方的立场上得出来的。

博弈树的特点

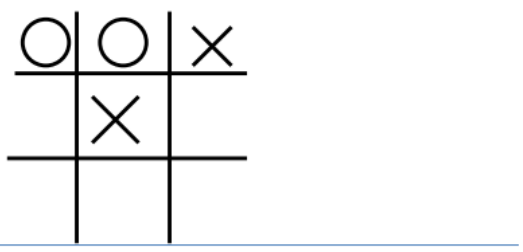
- 博弈树中的初始节点为博弈的开始格局
- 在博弈树中，“或”节点和“与”节点逐层交替出现。自己一方扩展的节点为“或”节点，对方扩展的节点为“与”节点。
- 所有能使自己一方获胜的终局都是本原问题，相应的节点是可解节点；所有使对方获胜的终局都是不可解节点。

极大极小分析法

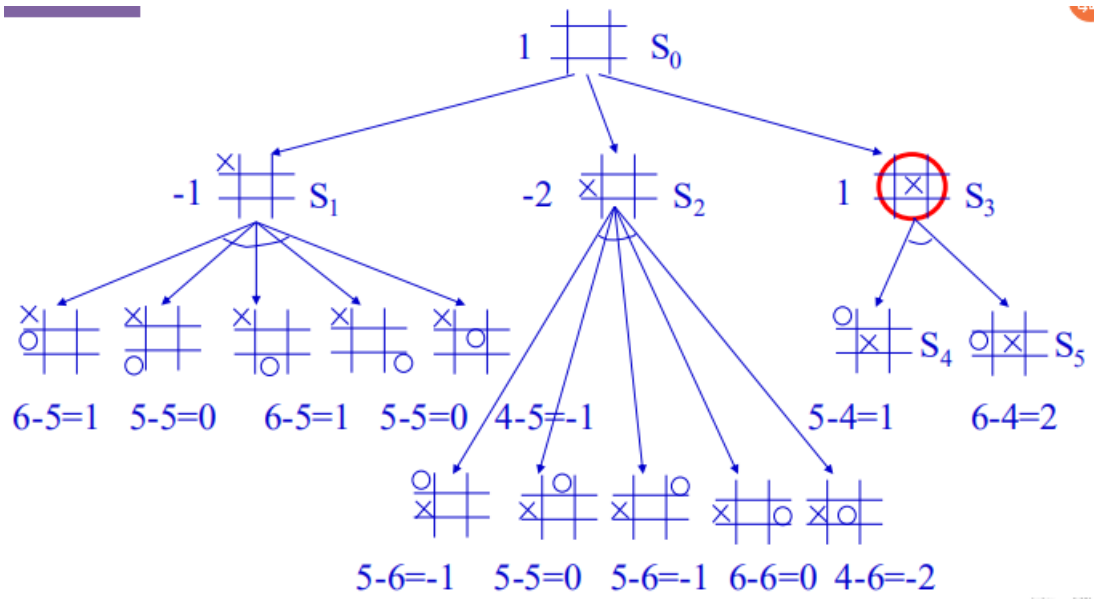
- 若父节点为“或”节点，则其分数等于其所有子节点分数的最大值
- 若父节点为“与”节点，则其分数等于其所有子节点分数的最小值。
- 计算出的父节点的分数值称为**倒推值**。
- 如果一个方案能获得较大的倒推值，则它就是当前最好的行动方案。

举例

win(x)表示x方的所有赢线数。如下图，win(A)=4, win(B)=2。



- 若n为非终局节点，则
- $h(n)=win(A)-win(B)$
- 若n为和局，则$h(n)=0$
- 若n为A取胜的终局节点，则$h(n)=+\infty$，如下图所示。
- 若n为A失败的终局节点，则$h(n)=-\infty$



α-β剪枝技术

- 一个“与”节点取当前子节点的最小值作为其倒推值的上界，称该值为β值。
- 一个“或”节点取当前子节点的最大值作为其倒推值的下界，称该值为α值。
  - 如果“或”节点x的α值不能降低其父节点的β值，即：

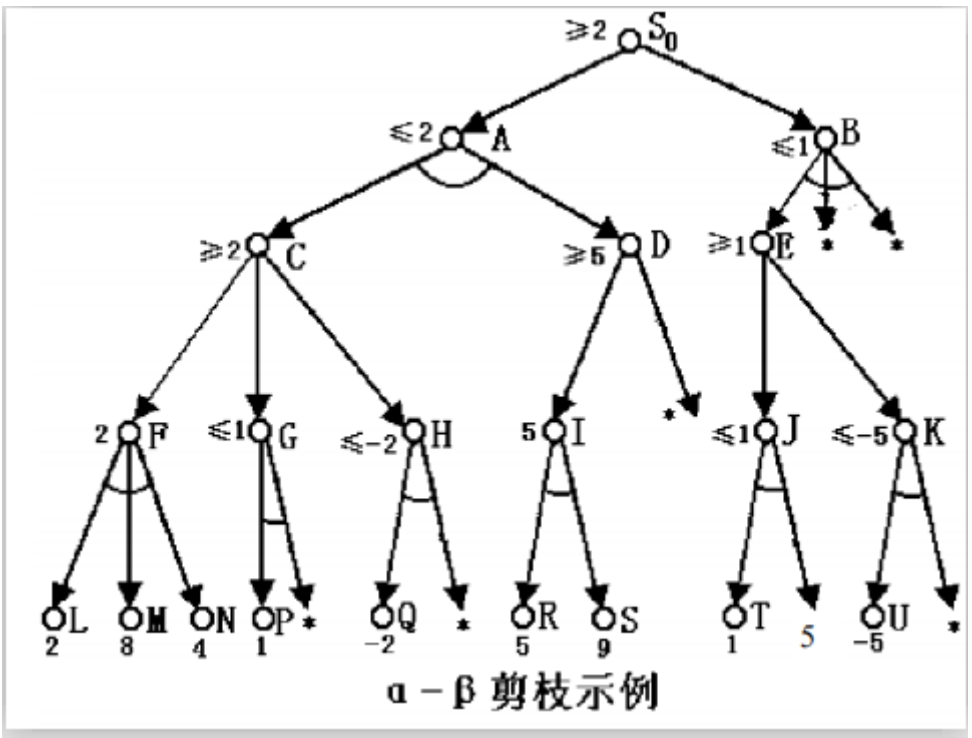
    $\alpha \geqslant \beta$

    则应停止搜索节点x的其余子节点，并使x的倒推值为α。这种技术称为β剪枝。

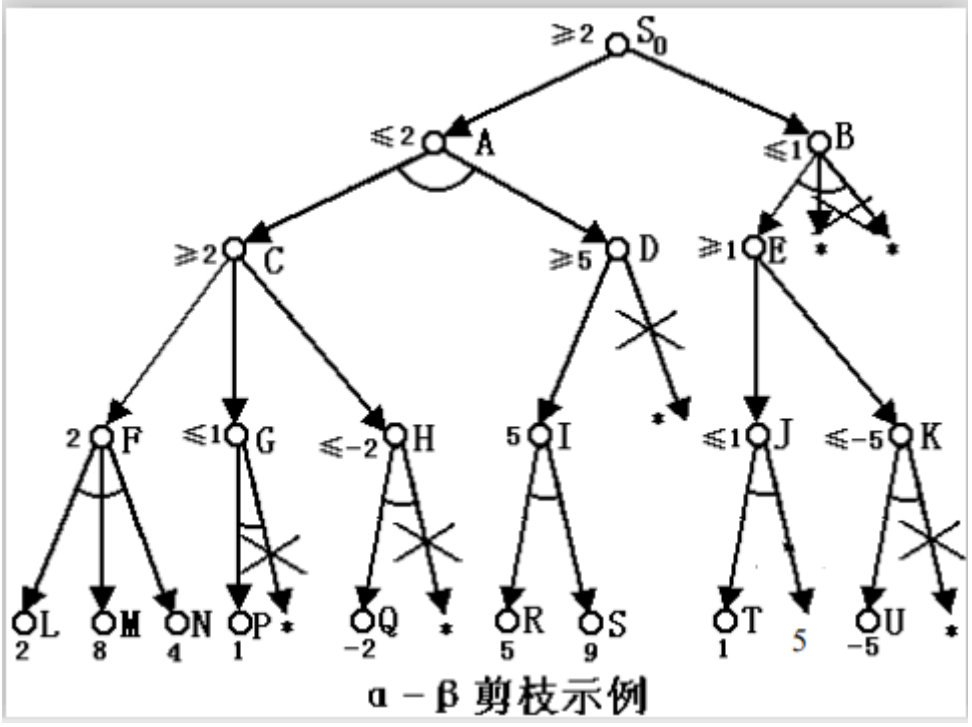
- 如果“与”节点x的β值不能升高其父节点的α值，即：

  $\beta \leqslant \alpha$

  则应停止搜索节点x的其余子节点，并使x的倒推值为β。这种技术称为α剪枝。



α－β 剪枝示例



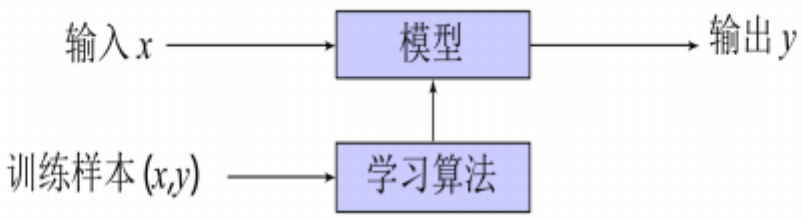
α－β 剪枝示例

15.机器学习朴素贝叶斯与决策树

机器学习

目前尚无统一定义

机器学习主要是研究如何使计算机从给定的数据中学习规律，即从观测数据（样本）中寻找规律，并利用学习到的规律（模型）对未知或无法观测的数据进行预测。目前，主流的机器学习算法是基于统计的方法，也叫统计机器学习



狭义地讲，机器学习是给定一些训练样本 $(x_i, y_i), 1 \le i \le N$（其中 x_i 是 输入，y_i 是需要预测的目标），让计算机自动寻找一个决策函数 $f(\cdot)$ 来建立 x 和 y 之间的关系。

$$\hat{y} = f(\Phi(x), \theta)$$

这里，$\hat{y}$是模型输出，θ为决策函数的参数，$\Phi(x)$表示样本 x 对应的特征表示。
因为 x 不一定都是数值型的输入，因此需要通过 $\Phi(x)$ 将 x 转换为数值型的输入。

回归

y 是连续值（实数或连续整数），f(x) 的输出也是连续值。这种类型的问题就是回归问题。对于所有已知或未知的(x,y)，使得f(x,θ) 和y 尽可能地一致。损函数通常定义为平方误差。

分类

y是离散的类别标记（符号），就是分类问题。损失函数有一般用 0-1 损失函数或负对数似然函数等。在分类问题中，通过学习得到的决策函数 f(x,θ) 也叫分类器。

有监督学习

是利用一组已知输入x和输出y 的数据来学习模型的参数，使得模型预测的输出标记和真实标记尽可能的一致

半监督学习

是利用少量已知输入x和输出y的数据以及未标注的样本，来学习模型的参数

无监督学习(UnsupervisedLearning）

用来学习的数据不包含标注信息，需要学习算法自动学习到一些有价值的信息，例如聚类(Clustering)

朴素贝叶斯

朴素贝叶斯分类假定一个属性值对给定类的影响独立于其他属性的值，即在属性间不存在依赖关系，也因此称为“朴素的”。

贝叶斯定理

$$P(C_i|X) = \frac{P(C_i \cap X)}{P(X)} = \frac{P(C_i) \times P(X|C_i)}{P(X)}$$

$P(C_i)$ **表示先验概率(Prior probability)。**

$P(C_i|X)$**表示后验概率(Posteriori probability)，先验概率是由以往的数据分析得到的。根据样本数据得到更多的信息后，对其重新修正，即是后验概率。**

朴素贝叶斯分类过程

1、每个数据样本用一个n维特征向量$X=\{x_1, x_2, \ldots, x_n\}$表示，分别描述对$n$个属性$A_1, A_2, \ldots, A_n$样本的$n$个度量。

2、假定有m个类$C_1, C_2, \ldots, C_m$。给定一个未知的数据样本X，分类法将预测X属于具有最高后验概率（条件X下）的类。即是说，朴素贝叶斯分类将未知的样本分配给类C_i，

当且仅当

$$P(C_i|X) > P(C_j|X), 1 \le j \le m, j \ne i$$

根据贝叶斯定理

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}$$

因此，由于$P(X)$对于所有类为常数，只需要$P(X|C_i)P(C_i)$最大即可。

3、假定属性值相互条件独立，即在属性间不存在依赖关系，这样，

$$P(X|C_i) = \prod_{k=1}^{n} P(x_k|C_i)$$

概率$P(x_k|C_i)$可以由训练样本估值，其中

（1）如果A_k是离散型属性，则$P(x_k|C_i) = s_{ik} / s_i$，其中$s_{ik}$是在属性$A_k$上具有$x_k$的类$C_i$的训练样本数，而$s_i$是$C_i$中的训练样本数。

（2）如果A_k是连续型属性，则通常假定该属性服从高斯分布。因而，

$$P(x_k \mid C_i)=g(x_k,\mu_{C_i},\sigma_{C_i})=\frac{1}{\sqrt{2\pi\sigma_{C_i}}}e\frac{(x_k-\mu_{C_i})^2}{2\sigma_{C_i}^2}$$

其中，给定类C_i的训练样本属性A_k的值，$g(x_k,\mu_{C_i},\sigma_{C_i})$是属性$A_k$的高斯密度函数。

4、为对未知样本X分类，对每个类C_i，计算$P(X \mid C_i)P(C_i)$。样本被指派到类C_i，当且仅当

$$P(X \mid C_i)P(C_i) > P(X \mid C_j)P(C_j)$$

即是说，X被指派到$P(X \mid C_i)P(C_i)$最大的类。

举例

朴素贝氏分类的实例

•办信用卡意愿：

项目	性别	年龄	学生身分	收入	办卡
1	男	>45	否	高	会
2	女	31~45	否	高	会
3	女	20~30	是	低	会
4	男	<20	是	低	不会
5	女	20~30	是	中	不会
6	女	20~30	否	中	会
7	女	31~45	否	高	会
8	男	31~45	是	中	不会
9	男	31~45	否	中	会
10	女	<20	是	低	会

判断：X=(女性，年龄介于31~45之间，不具学生身份，收入中等)会不会办理信用卡。

解：首先根据训练样本计算各属性相对于不同分类结果的条件概率：

•P(办卡)=7/10	P(不办卡)=3/10
•P(女性|办卡)=5/7	P(女性|不办卡)=1/3
•P(年龄=31~45|办卡)=3/7	P(年龄=31~45|不办卡)=1/3
•P(学生=否|办卡)=5/7	P(学生=否|不办卡)=0/3
•P(收入=中|办卡)=2/7	P(收入=中|不办卡)=2/3

其次，再应用朴素贝氏分类器进行类别预测：

•计算

•P(办卡)P(女性|办卡)P(年龄31~45|办卡)P(不是学生|办卡)P(收入中|办卡) =15/343≈0.044

•P(不办卡)P(女性|不办卡)P(年龄31~45|不办卡)P(不是学生|不办卡)P(收入中等|不办卡)=0

•0.044>0

其次，再应用朴素贝氏分类器进行类别预测：

•计算

•P(办卡)P(女性|办卡)P(年龄31~45|办卡)P(不是学生|办卡)P(收入中|办卡) =15/343≈0.044

•P(不办卡)P(女性|不办卡)P(年龄31~45|不办卡)P(不是学生|不办卡)P(收入中等|不办卡)=0

•0.044>0

2021-7-4 45

决策树

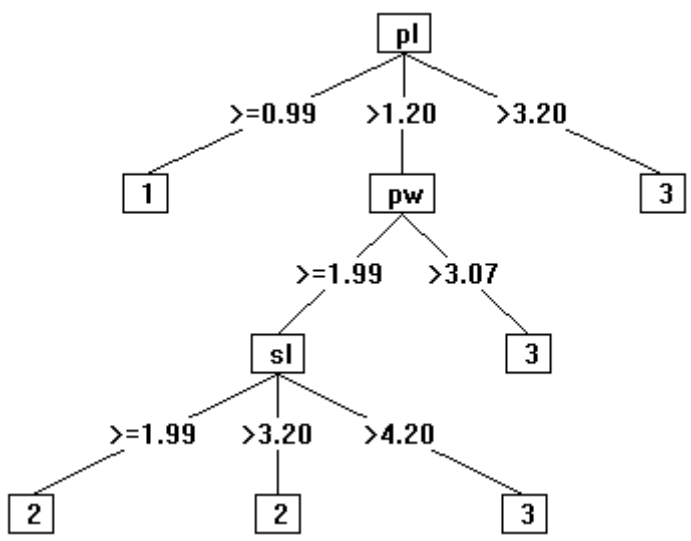
决策树算法是一种归纳分类算法,它通过对训练集的学习,挖掘出有用的规则,用于对新集进行预测。有监督的学习。

决策树归纳的基本算法是贪心算法，自顶向下递归方式构造决策树。

贪心算法：在每一步选择中都采取在当前状态下最好/优的选择。

在其生成过程中，分割方法即属性选择度量是关键。通过属性选择度量，选择出最好的将样本分类的属性。

决策树的结构



决策树算法以树状结构表示数据分类的结果。每个决策点实现一个具有离散输出的测试函数，记为分支。

根节点
非叶子节点（决策点）
叶子节点
分支

原理

基本算法（贪心算法）

- 自上而下分而治之的方法
- 开始时，所有的数据都在根节点
- 属性都是离散值字段 (如果是连续的，将其离散化)
- 所有记录用所选属性递归的进行分割
- 属性的选择是基于一个启发式规则或者一个统计的度量 (如, information gain)

停止分割的条件

- 一个节点上的数据都是属于同一个类别
- 没有属性可以再用于对数据进行分割

信息熵



ID3

> 简单分类器
>
> 集成分类器
>
> 分类：Naïve bayes、决策树、SVM、KNN、集成分类器（随机森林、bagging、boosting、多数投票）；分类器的评价与验证、机器学习流程。

感知器模型

感知器 是美国学者罗森勃拉特（Rosenblatt）于1957年为研究大脑的存储、学习和认知过程而提出的一类具有自学习能力的神经网络模型，其拓扑结构是一种分层前向网络。它包括：单层感知器和多层感知器。

单层感知器 是一种只具有单层可调节连接权值神经元的前向网络，这些神经元构成了单层感知器的输出层，是感知器的可计算节点。

在单层感知器中，每个可计算节点都是一个线性阈值神经元。当输入信息的加权和大于或等于阈值时，输出为1，否则输出为0或-1。

单层感知器的输出层的每个神经元都只有一个输出，且该输出仅与本神经元的输入及联接权值有关，而与其他神经元无关。

如果外部输入是线性可分的（指存在一个超平面可以将它们分开），则单层感知器一定能够把它划分为两类。其判别超平面由如下判别式确定：

$$\sum_{i=1}^{n} w_{ij} x_i - \theta_j = 0 \qquad j = 1,2,\ldots,m$$

作为例子，下面讨论用单个感知器实现逻辑运算的问题。事实上，单层感知器可以很好地实现“与”、“或”、“非”运算，但却不能解决“异或”问题。

多层感知器 是通过在单层感知器的输入、输出层之间加入一层或多层处理单元所构成的。其拓扑结构与图5.9所示的多层前向网络相似，差别也在于其计算节点的连接权值是可变的。

多层感知器的输入与输出之间是一种高度非线性的映射关系，如图4.9所示的多层前向网络，若采用多层感知器模型，则该网络就是一个从n维欧氏空间到m维欧氏空间的非线性映射。因此，多层感知器可以实现非线性可分问题的分类。例如，对“异或”运算，用图4.14所示的多层感知器即可解决。

支持向量机

决策边界离两类数据应尽可能远
最大化间隔 m

$$m = \frac{2}{||\mathbf{w}||}$$

关键思想:

为了解决非线性分割问题, 将 xi 变换到一个高维空间。
输入空间: xi 所在的空间
特征空间: 变换后 f(xi) 的空间

如何变换 ？

利用一个适当的变换f, 使分类变得容易些。
特征空间中的线性算子等价于输入空间中的非线性算子。

变换可能出现的问题

难以得到一个好的分类且计算开销大

SVM同时解决这两个问题

最小化 ||w||2 能得到好的分类
利用核函数技巧可以进行有效的计算

BP

BP神经网络（Back Propagation Neural Network），即误差后向传播神经网络，是一种按误差逆向传播算法训练的多层前馈网络，是目前应用最广泛的网络模型之一。

BP神经网络（Back Propagation Neural Network），即误差后向传播神经网络，是一种按误差逆向传播算法训练的多层前馈网络，是目前应用最广泛的网络模型之一。

> 与感知器模型不同的是，BP网络的激活函数f（·）要求是可微的，所以不能用二值函数，常用S型的对数、正切函数或线性函数。

BP算法由数据流的正向传播和误差信号的反向传播两个过程构成。

BP算法的实质是求取误差函数的最小值问题，这种算法采用的是非线性规划中的最速下降法，按误差函数的负梯度方向修改权值。



优点

1) 非线性映射能力
2) 自学习和自适应能力
3) 泛化能力
4) 容错能力

缺点

1) 局部极小化问题
2) BP 神经网络算法的收敛速度慢
3) BP 神经网络结构选择不一
4)　BP神经网络样本依赖性问题